

## LEARNER NOTES

# ML Module 1 — Traditional Machine Learning

From Machine Learning Fundamentals to Regularized Regression

For Data Science / ML consultants · Examples set in London · Python + scikit-learn

1. What is Machine Learning?
2. How Does ML Work?
3. Types of Machine Learning
4. Where Does Regression Fit?
5. Basic ML Workflow
6. Correlation Analysis
7. Correlation Matrix
8. From Correlation to Regression
9. Simple Linear Regression
10. How the Model Learns: Gradient Descent
11. Python: Simple Linear Regression
12. Multiple Linear Regression
13. Model Evaluation
14. Actual vs Predicted
15. Generalization
16. Underfitting & Overfitting
17. Bias and Variance
18. Why Regularization?
19. Ridge Regression
20. Lasso Regression
21. Ridge vs Lasso
22. Hands-on Lab
23. Model Comparison
24. ML Module 1 Recap

## START HERE

01

# What is Machine Learning?

Let's start with a question every client eventually asks: "*Can we predict this?*" Machine Learning is how we answer it — not by writing rules, but by learning from examples.

## A simple example: London flat prices

| HOUSE SIZE       | PRICE    |
|------------------|----------|
| 500 sq ft        | £300k    |
| 750 sq ft        | £450k    |
| 1,000 sq ft      | £600k    |
| <b>875 sq ft</b> | <b>?</b> |

### 🧠 What does your brain do?

You look at the examples, spot a pattern (*every extra 250 sq ft adds about £150k — roughly £600 per sq ft*), and guess the missing price — roughly **£525k**.

That is exactly what Machine Learning does — just with thousands of rows and many more columns.



### CORE IDEA

Machine Learning learns patterns from existing data and uses those patterns to make predictions or decisions on new data.

## Traditional Programming vs Machine Learning

### Flip between the two approaches

#### Traditional Programming



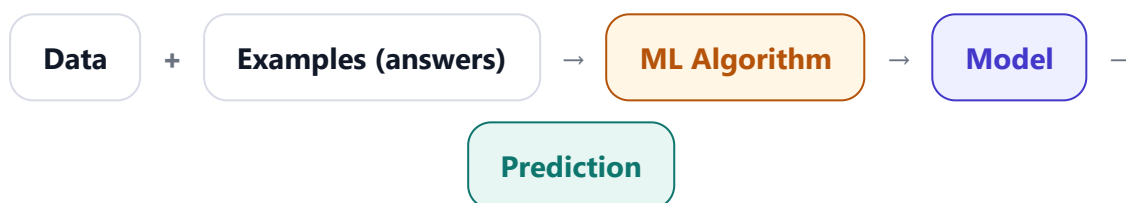
A developer **writes the rule** by hand:

```

if area > 1200: price = "high"
price = area × £600      # rule someone had to figure out
  
```

Works when rules are known and stable. Breaks when the pattern is complex (a London price depends on area, borough, Tube access, age, schools, demand...) or keeps changing.

#### Machine Learning



We give the algorithm **past examples with known answers**. It **figures out the rule itself** and stores it as a *model*.

Works when the pattern is too complex to hand-code, or when we have lots of historical data. The "rule" is learned, not written.

|                               | TRADITIONAL PROGRAMMING    | MACHINE LEARNING                  |
|-------------------------------|----------------------------|-----------------------------------|
| <b>Who creates the logic?</b> | The developer              | The algorithm, from data          |
| <b>Input</b>                  | Rules + data               | Data + known answers              |
| <b>Output</b>                 | Answers                    | A model (the learned rules)       |
| <b>Best for</b>               | Tax calculation, invoicing | Price prediction, fraud detection |



#### LEADING TO THE NEXT CONCEPT

*"Now that we understand what ML is, the next question is: how does a machine actually learn from data?"*

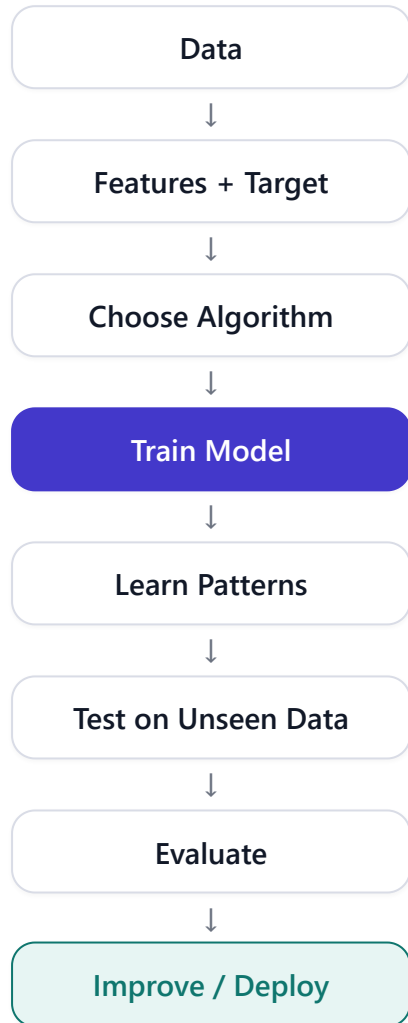
## THE LEARNING CYCLE

02

# How Does Machine Learning Work?

**WHY ARE WE LEARNING THIS NOW?**

We know *what* ML is. Before touching any algorithm, we need the big picture of *how* learning happens — every model today follows this cycle.



Click any step to see what happens there.

Start with **Data** — historical records where we already know the answer.

**Two words you'll hear throughout this module**

**Features (X)** — the inputs we use to predict.  
*e.g. experience*

**Target (y)** — the value we want to predict. *e.g. salary*

## Example: predicting a London analyst's salary from experience

Try it: change experience, watch the prediction

INTUITION DEMO — NOT A REAL TRAINED MODEL

| EXPERIENCE | SALARY |
|------------|--------|
| 1 year     | £30k   |
| 3 years    | £42k   |
| 5 years    | £55k   |
| 8 years    | £75k   |

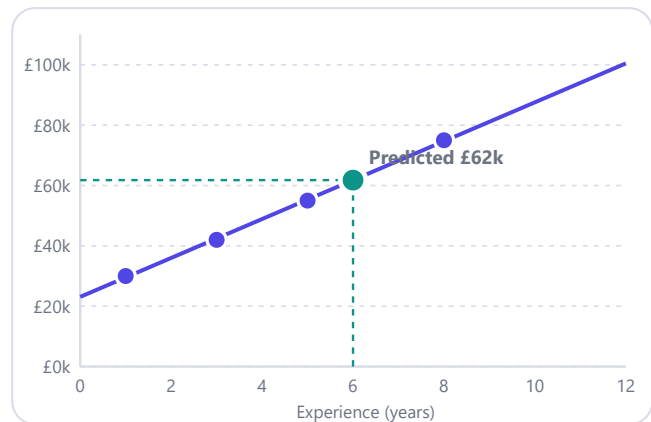
6 yrs

→

Model

→

£62k



The line is the "pattern" learned from the 4 examples. A new input lands on that line → that's the prediction. Notice how predictions far outside 1–8 years (e.g. 12 years) are less trustworthy — the model never saw such data.

 Sketch pad — explain "How Does ML Work?" by hand

saved notes

| I/p feature $\rightarrow$ | Salary $\rightarrow$ Target |
|---------------------------|-----------------------------|
| 5                         | 20k                         |
| 7                         | 35k                         |
| 9                         | 50k                         |
| 3                         | 15k                         |
| 15 = ?                    |                             |

Exp, Sal  $\rightarrow$  

(5, 20)



**LEADING TO THE NEXT CONCEPT**

"We've seen a model learn from labelled examples. But is that the only way machines learn?  
Let's look at the different types of ML."

## THE ML FAMILY

03

# Types of Machine Learning

**WHY ARE WE LEARNING THIS NOW?**

Not every problem has "known answers" in the data. The type of data we have decides the type of ML we use.

👉 Click a card to explore each type.

**Supervised**

Learn from data with known answers

**Unsupervised**

Find hidden structure — no answers given

**Reinforcement**

Learn by trial, reward and penalty

**Supervised Learning****THIS MODULE**

Input + Known Output →

Learn →

**Predict Output**

Like a student with a solved answer key: the data contains the **target column**, so the model can check itself while learning.

**Regression****This module's focus**

Predict a **number** (continuous value)

House price · Salary · Sales · Demand

**Classification**

Predict a **category** (label)

Spam / Not spam · Fraud / Not fraud ·  
Disease / No disease



**⚡ CHECK 1**

**Predicting house price is:**

A. Classification

**B. Regression ✓ correct**

C. Clustering

D. Reinforcement Learning

**Answer: B. Regression** — House price is a continuous number (£525k, £650k...), and we have past prices to learn from → supervised regression.

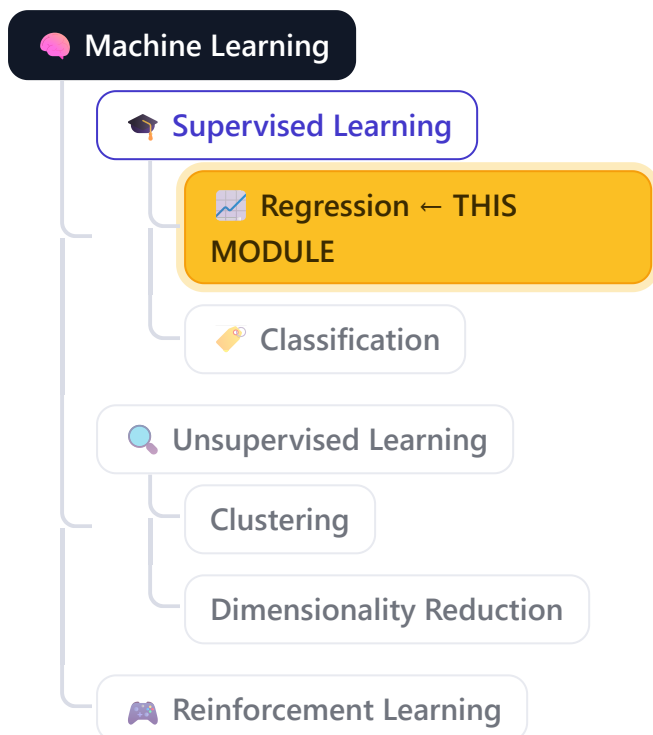
**LEADING TO THE NEXT CONCEPT**

*"Now we know regression lives inside supervised learning. Let's zoom in and see exactly where it sits on the ML map."*

## THE MAP

04

## Where Does Regression Fit?



## DEFINITION

Regression is a supervised learning problem where the target is a continuous numerical value.

| INPUT                      | REGRESSION OUTPUT |
|----------------------------|-------------------|
| Flat in Islington          | £650k             |
| Data consultant            | £75k salary       |
| High-street store (weekly) | 10,500 sales      |
| Tube station (daily)       | 3,200 passengers  |



## QUICK TEST

Ask: "Is the answer a number on a scale, or a label from a list?" Number → Regression. Label → Classification.

**Sketch pad — explain “Where Does Regression Fit?” by hand****saved notes****LEADING TO THE NEXT CONCEPT**

*“Now we know where Regression fits. Before building anything, let's see the end-to-end workflow a real ML project follows.”*

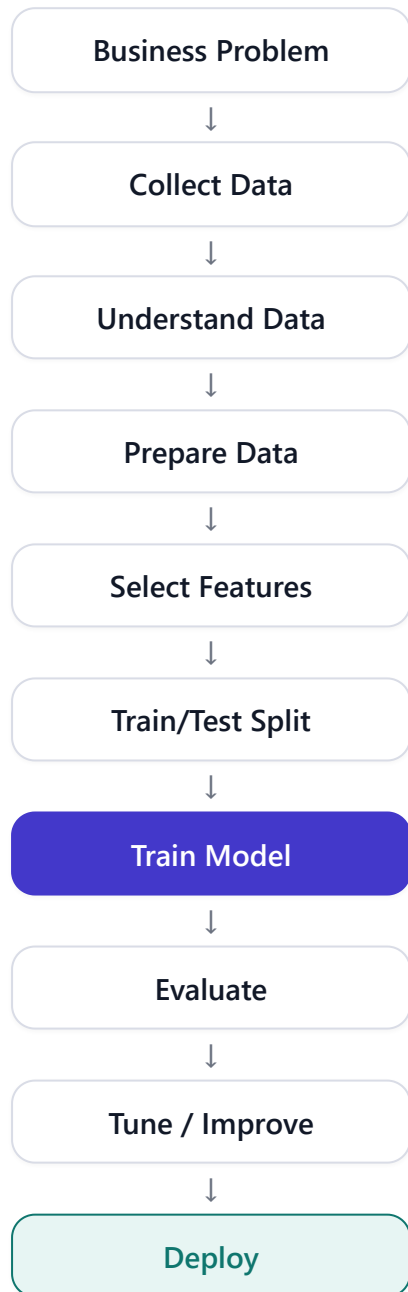
## END-TO-END

05

# Basic Machine Learning Workflow

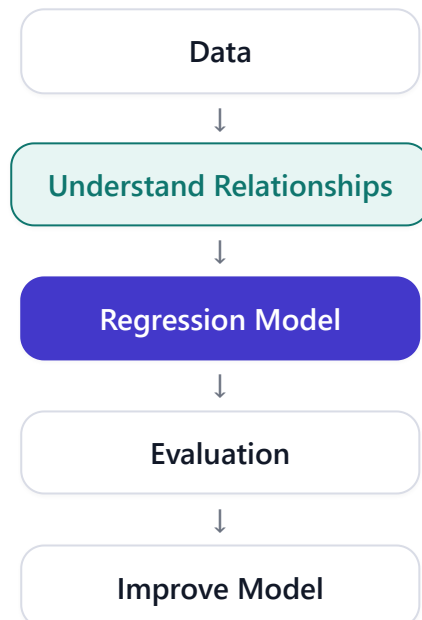
**WHY ARE WE LEARNING THIS NOW?**

A model is only one step. In consulting, most time goes into everything *around* the model. This is the route map for every project.

**A real project**

Click a step for a one-line explanation.

Every project starts with a **business problem**: "Can we estimate a fair price for a listing before it goes live?"

**This module's zoomed-in workflow**

**WHY UNDERSTAND DATA BEFORE MODELLING?**

A model can only learn what's in the data. If we don't know which columns relate to the target, which ones overlap, or which ones are junk, we'll build a model we can't explain or trust.

**Garbage in → garbage out.**

**LEADING TO THE NEXT CONCEPT**

*"Before building a regression model, let's first understand the relationships between our variables."*

## RELATIONSHIPS

06

# Correlation Analysis

**WHY ARE WE LEARNING THIS NOW?**

Before building a regression model, we first need to understand the relationships inside our data. Correlation is our first lens.

**DEFINITION**

Correlation measures the strength and direction of a linear relationship between two numerical variables.

**Positive**

Experience  $\uparrow$   $\rightarrow$  Salary  $\uparrow$   
Both move together

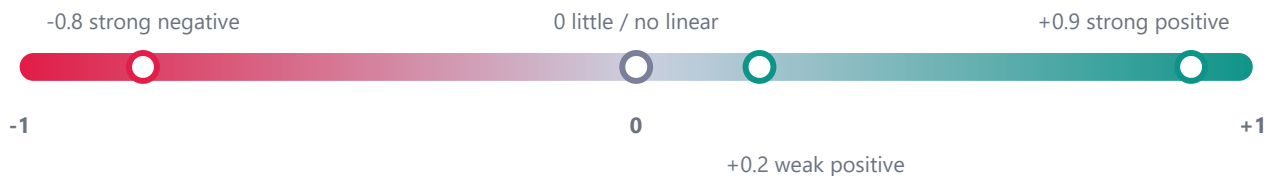
**Negative**

Distance from central London  $\uparrow$   $\rightarrow$  Price  $\downarrow$   
One goes up, the other goes down

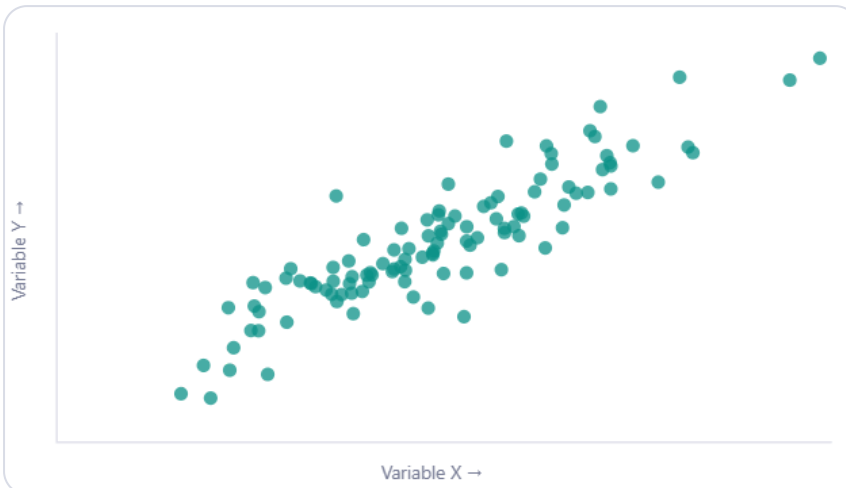
**None / weak**

Employee ID  $\leftrightarrow$  Salary  
No consistent pattern

## The correlation coefficient (r)



Sign (+/-) = direction. Distance from 0 = strength. A rough guide:  $|r| > 0.7$  strong, 0.3–0.7 moderate,  $< 0.3$  weak.

**Interactive: see correlation change the shape****SIMULATED DATA****CORRELATION R (COMPUTED)****+0.90**

**strong positive** linear relationship. As X rises, Y tends to rise.

**⚡ CHECK 2****Which correlation represents the strongest positive relationship?**

A. -0.92

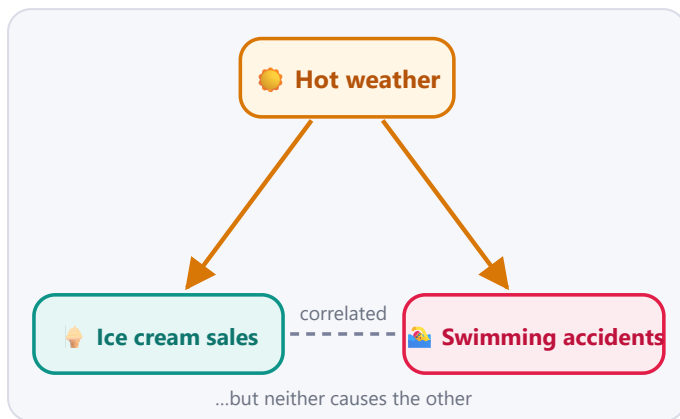
B. 0.12

**C. 0.91 ✓ correct**

D. 0.01

**Answer: C. 0.91** — 0.91 is closest to +1, so it's the strongest positive relationship. -0.92 is strong too — but negative.

## ⚠️ Correlation does not imply causation



During a UK summer heatwave, ice cream sales and swimming accidents rise together in the data. Does ice cream cause drowning? Of course not.

A **third variable — hot weather** — drives both. More people buy ice cream *and* more people go swimming.



**Correlation tells us two things move together. It does not tell us one causes the other.** Always ask: "Is there a hidden third factor?"



### LEADING TO THE NEXT CONCEPT

*"Correlation compares two variables at a time. But real datasets have many columns. How do we see all the relationships at once?"*

## 07

## MANY VARIABLES AT ONCE

## Correlation Matrix



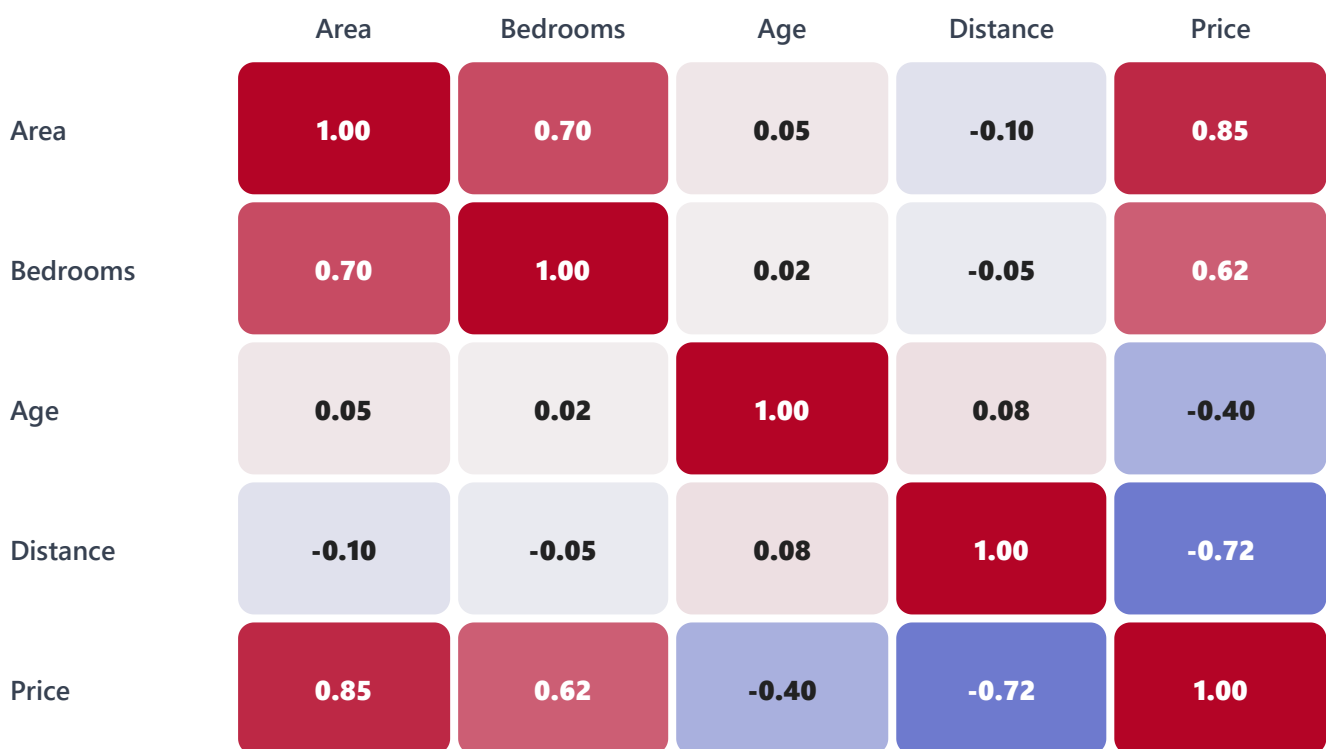
## WHY ARE WE LEARNING THIS NOW?

With 5 columns there are 10 pairs to check; with 20 columns, 190. A **correlation matrix** shows every pair in one grid — and a heatmap makes it readable at a glance.

## Example housing correlation heatmap

## ILLUSTRATIVE VALUES

👉 Click any cell to read what it means.



-1 strong negative

0 none

+1 strong positive

Click a cell...

**Strong +**

Area ↔ Price (0.85)  
— useful predictor

**Strong -**

Distance ↔ Price  
(-0.72) — also  
useful

**Weak**

Age ↔ Area (0.05)  
— little linear link

**Diagonal = 1**

Every variable  
correlates perfectly  
with itself

## Correlation table

PYTHON

```
df.corr(numeric_only=True)
```

Returns a table of r values for every pair of numeric columns. `numeric_only=True` skips text columns like `location`.

## Correlation heatmap

PYTHON

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.show()
```

`annot=True` writes the number in each cell; `cmap="coolwarm"` makes negatives blue and positives red.

**USE IT WISELY**

Correlation can help us understand relationships and identify potentially useful or **redundant** features (e.g. Area ↔ Bedrooms = 0.70 — they carry overlapping information), but it should **not** be treated as the only feature-selection method. It only sees *linear, pairwise* relationships.

**LEADING TO THE NEXT CONCEPT**

*"We have looked at relationships between variables. Now let's use those relationships to build a model that can make predictions."*

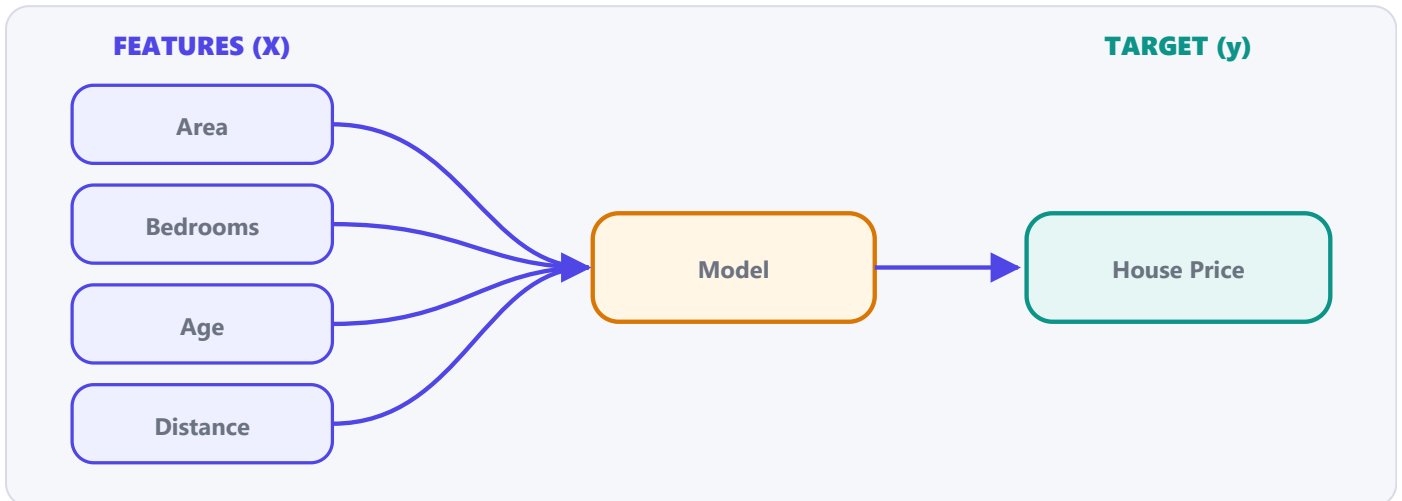
## THE BRIDGE

08

# From Correlation to Regression

**WHY ARE WE LEARNING THIS NOW?**

We have looked at relationships between variables. Now let's use those relationships to build a model that can make predictions. Correlation *describes*; regression *predicts*.

**X = Features**

The columns we **use** to make a prediction. Capital X because it's usually a table (many columns).

**y = Target**

The single column we **want to predict**. Lower-case y because it's one column.

**Your turn: we want to predict the selling price. Click the target (y).**

area

bedrooms

age

distance

price

Which column is the answer we want the model to produce?

**LEADING TO THE NEXT CONCEPT**

*"Let's start with the simplest possible model: one feature, one target, one line."*

## ONE FEATURE, ONE LINE

09

# Simple Linear Regression

**DEFINITION**

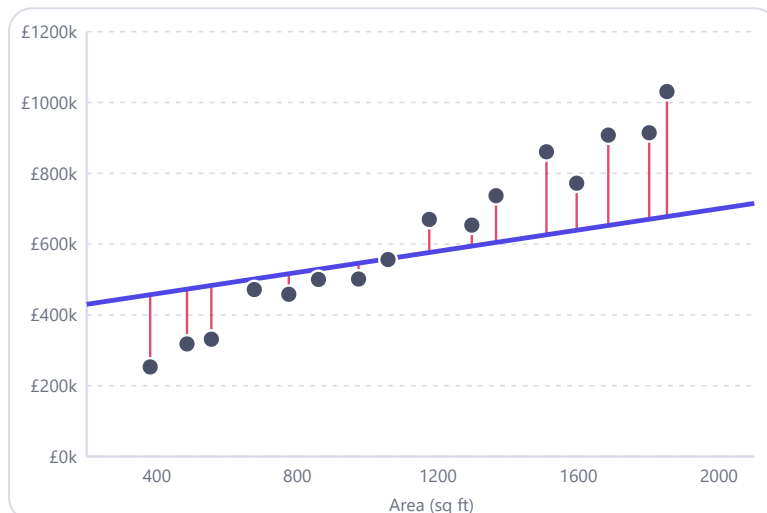
Simple Linear Regression uses one feature to predict one continuous target.

Area



House Price

Picture the data as dots on a chart. **The model tries to find a line that best represents the relationship between the input and target.** "Best" simply means: the line that stays as close as possible to all the dots at once — the one that makes the smallest overall prediction errors.

**Be the algorithm: find the best line yourself****SAMPLE DATA****YOUR AVG. ERROR****£136k****BEST POSSIBLE****?**☒ Show errors (residuals)

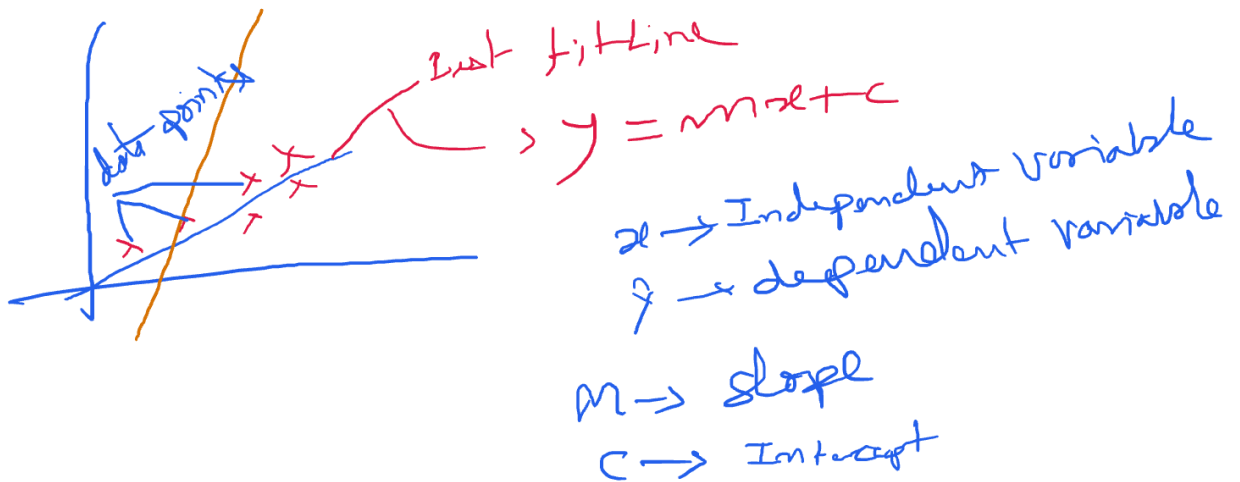
Move the sliders and try to shrink the red error lines.



That's exactly what `LinearRegression().fit()` does — it tries every slope/starting point (cleverly, in one shot) and keeps the line with the smallest total error. You just did it by hand!

 Sketch pad — explain “Simple Linear Regression” by hand

saved notes



## LEADING TO THE NEXT CONCEPT

“But how does the computer actually find the best line? It can't see the chart — so let's watch it learn, step by step.”

## UNDER THE HOOD

10

# How the Model Learns: Gradient Descent

**WHY ARE WE LEARNING THIS NOW?**

You just found a good line by moving sliders and watching the error. But **how does the computer move the sliders?** It can't "see" the chart. It uses a simple repeat-until-good recipe called **Gradient Descent** — the same idea that trains almost every modern ML model, including neural networks.

## The intuition: walking downhill in the fog



Imagine you're on a hillside in thick fog and want to reach the lowest point of the valley. You can't see far — but you **can feel which way the ground slopes under your feet**.

So you take a small step downhill. Feel the slope again. Another step. Repeat until the ground is flat.

The **height** = the model's error (MSE). Your **position** = the line's slope and starting point. The **slope under your feet** = the *gradient*. Your **step size** = the *learning rate*.

Start with any line ( $m = 0, c = 0$ )



Predict prices with the current line



Measure the error (MSE)



Find the downhill direction (gradient)



Take a small step (learning rate)



Repeat until the error stops falling

## The recipe (just five lines)

Our line is **price =  $m \times \text{area} + c$**  —  **$m$**  is the slope (steepness) and  **$c$**  is the intercept (starting point). Gradient descent nudges both, again and again:

| STEP        | FORMULA                                                         | IN PLAIN ENGLISH                                                                  |
|-------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1. Predict  | $\hat{y} = m \cdot x + c$                                       | Use the current line to guess every price                                         |
| 2. Error    | $e = \hat{y} - y$                                               | How far off is each guess? (+ too high, – too low)                                |
| 3. Cost     | $MSE = \text{average}(e^2)$                                     | One number for "how bad is this line?"                                            |
| 4. Gradient | $dm = (2/n) \cdot \sum(e \cdot x)$<br>$dc = (2/n) \cdot \sum e$ | Which way, and how strongly, should m and c change to reduce the cost?            |
| 5. Update   | $m = m - \alpha \cdot dm$<br>$c = c - \alpha \cdot dc$          | Step <i>against</i> the gradient (downhill). $\alpha$ = learning rate = step size |



Why the minus sign? The gradient points **uphill** (the direction that increases error). We want less error, so we step the **opposite** way.

## Watch it learn — London flats

Three tiny training examples, kept small so we can do the maths by hand. Area is in **100s of sq ft** and price in **£100k**:



### Flat A

400 sq ft → £300k

$x = 4, y = 3$



### Flat B

600 sq ft → £400k

$x = 6, y = 4$



### Flat C

800 sq ft → £500k

$x = 8, y = 5$

The "true" answer is  $m = 0.5, c = 1$  (each extra 100 sq ft adds £50k, plus a £100k starting point). The model doesn't know that — it starts from a flat line at zero and has to discover it.

## Gradient descent, step by step

LIVE CALCULATION

STEP

**0**

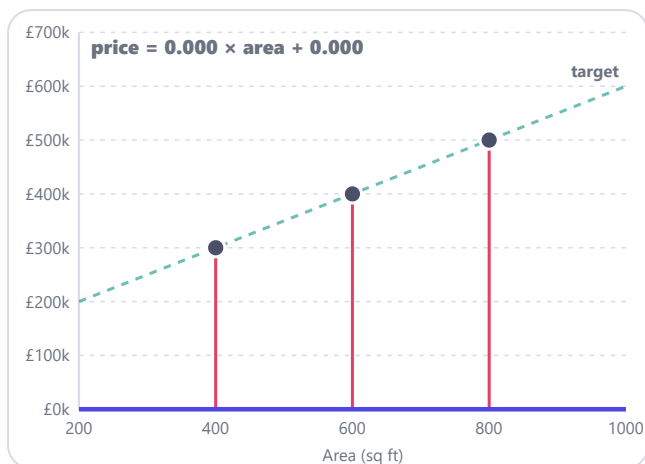
SLOPE M

**0.000**

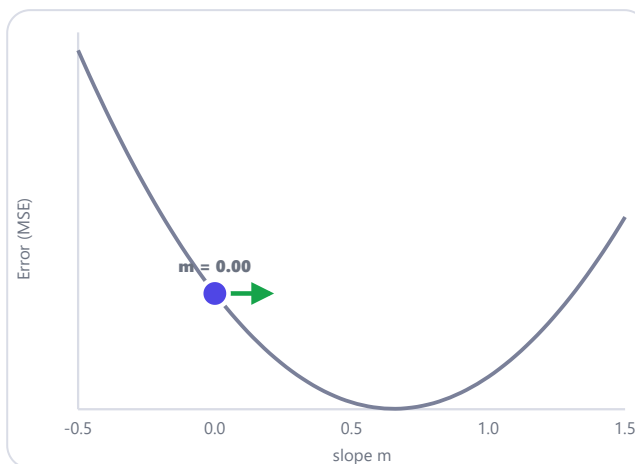
INTERCEPT C

**0.000**

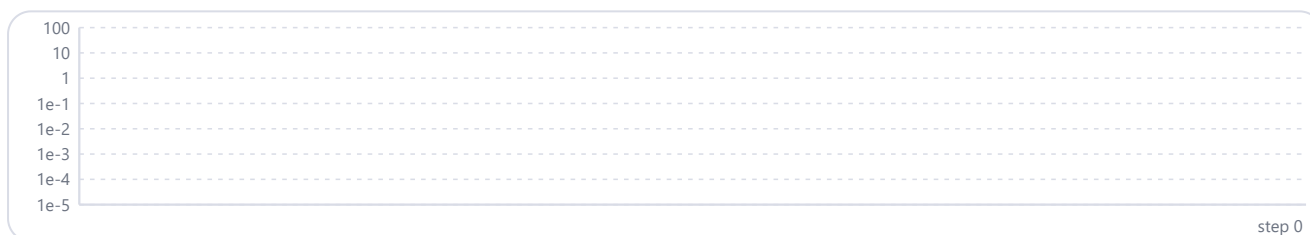
ERROR (MSE)

**16.6667**

The line being learned · red = errors



The "valley": error for every possible slope m (c held at its current value) · ● = where we are



Error (MSE) after each step — log scale, so every drop is visible

Start: a flat line at zero. Every prediction is £0 — the error is huge (MSE 16.67). Press **► One step** to see the first update.



## The maths for the next (step 1) step

| FLAT       | X | Y | $\hat{Y} = 0.000 \cdot X + 0.000$ | $E = \hat{Y} - Y$ | $E^2$          | $E \cdot X$    |
|------------|---|---|-----------------------------------|-------------------|----------------|----------------|
| A          | 4 | 3 | 0.000                             | -3.000            | 9.0000         | -12.000        |
| B          | 6 | 4 | 0.000                             | -4.000            | 16.0000        | -24.000        |
| C          | 8 | 5 | 0.000                             | -5.000            | 25.0000        | -40.000        |
| <b>Sum</b> |   |   |                                   | <b>-12.000</b>    | <b>50.0000</b> | <b>-76.000</b> |

**Cost & gradients**

$$\text{MSE} = 50.0000 / 3 = 16.6667$$

$$dm = (2/3) \times (-76.000) = -50.667$$

$$dc = (2/3) \times (-12.000) = -8.000$$

**Update ( $\alpha = 0.01$ )**

$$m = 0.000 - 0.01 \times (-50.667) = 0.507$$

$$c = 0.000 - 0.01 \times (-8.000) = 0.080$$

dm is negative  $\rightarrow$  m goes up · dc is negative  
 $\rightarrow$  c goes up

**Step 1 by hand — so you can check it on the whiteboard**

Starting from  $m = 0$ ,  $c = 0$ , learning rate  $\alpha = 0.01$ :

| FLAT       | X | Y | $\hat{Y} = 0 \cdot X + 0$ | $E = \hat{Y} - Y$ | $E \cdot X$ |
|------------|---|---|---------------------------|-------------------|-------------|
| A          | 4 | 3 | 0                         | -3                | -12         |
| B          | 6 | 4 | 0                         | -4                | -24         |
| C          | 8 | 5 | 0                         | -5                | -40         |
| <b>Sum</b> |   |   |                           | <b>-12</b>        | <b>-76</b>  |

**Cost**

$$\text{MSE} = (9 + 16 + 25) / 3 = 16.67$$

**Gradients**

$$dm = (2/3) \times (-76) = -50.67$$

$$dc = (2/3) \times (-12) = -8.00$$

Both negative  $\rightarrow$  increasing m and c would *reduce* the error.

**Update**

$$m = 0 - 0.01 \times (-50.67) = 0.507$$

$$c = 0 - 0.01 \times (-8.00) = 0.080$$

After just **one** step, the MSE drops from **16.67**  
 $\rightarrow$  **0.77**. 🎉

Press **► One step** above and you'll see exactly these numbers.

**The learning rate: the most important knob****Too small (0.002)**

Safe, but painfully slow — needs many more steps to reach the bottom.

**Just right (0.01)**

Big early drops, then steady progress to the bottom.

**Too big (0.03)**

Overshoots the valley, lands higher up the other side, and the error **explodes**. Try it above!

**NOTICE SOMETHING?**

The slope `m` gets close in 2–3 steps, but the intercept `c` crawls slowly (it takes thousands of steps to reach 1.0). That's because `x` values (4–8) are much bigger than 1, so the two "directions" of the valley have very different steepness. In practice we **scale features** (put them on similar ranges) so gradient descent converges much faster.

## Gradient descent in Python (from scratch)

### Gradient descent for simple linear regression

PYTHON

```
import numpy as np

x = np.array([4, 6, 8])      # area in 100s of sq ft (400, 600, 800 sq ft)
y = np.array([3, 4, 5])      # price in £100k (£300k, £400k, £500k)

m, c = 0.0, 0.0              # start with a flat line
lr = 0.01                     # learning rate
n = len(x)

for step in range(1, 5001):
    y_pred = m * x + c        # 1. predict
    error = y_pred - y        # 2. error
    mse = (error ** 2).mean()  # 3. cost
    dm = (2 / n) * (error * x).sum()  # 4. gradients
    dc = (2 / n) * error.sum()
    m -= lr * dm               # 5. update (step downhill)
    c -= lr * dc
    if step in (1, 2, 3, 10, 100, 1000, 5000):
        print(f"step {step:>4}: m={m:.3f} c={c:.3f} MSE={mse:.4f}")
```

### ▼ Expected output

```
step    1: m=0.507 c=0.080 MSE=16.6667
step    2: m=0.612 c=0.098 MSE=0.7745
step    3: m=0.634 c=0.102 MSE=0.0867
step   10: m=0.638 c=0.112 MSE=0.0546
step  100: m=0.622 c=0.213 MSE=0.0428
step 1000: m=0.536 c=0.766 MSE=0.0038
step 5000: m=0.500 c=0.999 MSE=0.0000
```

It finds `m = 0.5` and `c = 1.0` — exactly the true pattern. (The MSE printed is the cost *before* that step's update.)

**HOW DOES THIS RELATE TO SCIKIT-LEARN?**

`LinearRegression()` actually uses a direct formula (ordinary least squares) that jumps straight to the bottom of the valley — perfect for small/medium tables. `SGDRegressor()` trains the same kind of model with (stochastic) gradient descent, which shines when data is huge. And every neural network you'll meet later is trained with gradient descent. Same recipe, much bigger valley.

**⚡ KNOWLEDGE CHECK**

**In gradient descent, what does the learning rate control?**

A. How many features the model uses

B. The number of training rows

**C. How big a step we take downhill on each update ✓ correct**

D. The train/test split ratio

**Answer: C. How big a step we take downhill on each update** — The learning rate is the step size. Each update moves  $m$  and  $c$  by  $\text{learning rate} \times \text{gradient}$ .

**⚡ KNOWLEDGE CHECK**

During training, the MSE gets bigger with every step. What's the most likely fix?

A. Increase the learning rate

**B. Decrease the learning rate ✓ correct**

C. Add more test data

D. Switch to classification

**Answer: B. Decrease the learning rate** — Error that grows step after step means the model is overshooting the valley. Reduce the learning rate (and consider scaling features).

**LEADING TO THE NEXT CONCEPT**

*"Now that we know how the line is learned, let's ask Python to train it for us."*

## HANDS-ON

11

# Python: Simple Linear Regression

**WHY ARE WE LEARNING THIS NOW?**

We understand the idea. scikit-learn turns it into 5 lines of code — and the same pattern ( `fit` → `predict` ) works for almost every model you'll use.

## Simple Linear Regression with scikit-learn

PYTHON

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[["area"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

`X = df[["area"]]` Double brackets keep X as a **table** (DataFrame) — scikit-learn always expects a 2-D input, even for one feature.

`y = df["price"]` Single brackets → one column, the target.

`train_test_split( ... )` Hides 20% of rows as a **test set** the model never sees during training — our "exam paper".

`random_state=42` Makes the split repeatable, so everyone in the room gets the same numbers.

`model.fit(X_train, y_train)` Learning happens here — the model finds the best-fit line.

`model.predict(X_test)` Uses the learned line to predict prices for unseen houses.

**▼ Peek inside the model (optional)**

## What did the model learn?

PYTHON

```
print(model.coef_)      # how much price changes per extra sq.ft  
print(model.intercept_) # where the line starts
```

Price is in £ thousands, so a coefficient of 0.45 would mean "each extra sq ft adds about £0.45k (£450) to the price, on average."

**LEADING TO THE NEXT CONCEPT**

*"Is house price determined only by area? Of course not. Let's give the model more information."*

## MORE FEATURES

12

## Multiple Linear Regression



## THE QUESTION

Is house price determined only by area?

Two 900 sq ft flats — one a new-build next to a Zone 1 Tube station, one a 100-year-old conversion in Zone 6, 15 miles out — won't cost the same. Our one-feature model can't see that difference.

Area · Bedrooms · Age · Distance · Borough



Model



House Price



## DEFINITION

Multiple Linear Regression uses multiple features to predict a continuous target.

## Simple Linear Regression

One feature



Target

Area → Price

## Multiple Linear Regression

Many features



Target

Area + Bedrooms + Age + Distance → Price

## Multiple Linear Regression

PYTHON

```
x = df[
    [
        "area",
        "bedrooms",
        "age",
        "distance"
    ]
]

y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

The only change is **which columns go into X**. Each feature now gets its own coefficient — its own "weight" in the prediction. (We re-split because X changed.)

#### ⚡ KNOWLEDGE CHECK

**Which of the following is Multiple Linear Regression?**

A. Predicting salary using only years of experience

B. Predicting spam / not-spam using email words

**C. Predicting house price using area, bedrooms, age and distance ✓ correct**

D. Predicting sales using only advertising budget

**Answer: C. Predicting house price using area, bedrooms, age and distance** — Multiple Linear Regression = several features → one continuous target. (B) is classification; (D) has one feature, so it's simple linear regression.



Sketch pad — explain "Multiple Linear Regression" by hand

saved notes

$$y = m_1 x_1 + m_2 x_2 + \dots + m_n x_n + c$$

A, B, D, Age

$$y = m_1 A + m_2 B + m_3 D + m_4 \text{Age} + c$$

**LEADING TO THE NEXT CONCEPT***"We now have predictions. But how do we know whether they are good?"*

## IS IT ANY GOOD?

13

# Model Evaluation

**WHY ARE WE LEARNING THIS NOW?**

We now have predictions. But how do we know whether they are good? A client will always ask: "How accurate is it?" — these metrics are the answer.

**MAE — Mean Absolute Error**

*"On average, how far are predictions from actual values?"*

MAE = £30k → typical miss is about £30k. Easy to explain to a business user.

**MSE — Mean Squared Error**

*"Large errors are penalized more heavily because errors are squared."*

A £50k miss counts 25× as much as a £10k miss, not 5×. Units are "(£k)<sup>2</sup>", so it's hard to read directly.

**RMSE — Root Mean Squared Error**

*"Error on the target's scale, with extra weight on big misses."*

Back in £k. If RMSE is much bigger than MAE, a few predictions are badly off.

**R<sup>2</sup> — R-squared**

*"How well does the model explain variation in the target, compared to a baseline that always predicts the average?"*

1.0 = perfect · 0 = no better than guessing the average · can go negative if worse.

### Live metric calculator — edit a prediction and watch the metrics react

| PROPERTY                                                                                              | ACTUAL (£K) | PREDICTED (£K)                   | ERROR |
|-------------------------------------------------------------------------------------------------------|-------------|----------------------------------|-------|
|  Croydon flat        | 420         | <input type="text" value="450"/> | -30.0 |
|  Hackney flat        | 515         | <input type="text" value="490"/> | +25.0 |
|  Walthamstow terrace | 600         | <input type="text" value="630"/> | -30.0 |
|  Richmond terrace    | 735         | <input type="text" value="705"/> | +30.0 |
|  Clapham house       | 880         | <input type="text" value="900"/> | -20.0 |

MAE

**£27.0k**

MSE

**745 (£k)<sup>2</sup>**

RMSE

**£27.3k**R<sup>2</sup>**0.972**

Notice: one big miss moves RMSE much more than MAE. That's the "squaring" effect.

| METRIC               | MAIN QUESTION                                     | BETTER WHEN |
|----------------------|---------------------------------------------------|-------------|
| <b>MAE</b>           | How far off are predictions on average?           | Lower ↓     |
| <b>MSE</b>           | How strongly are large errors penalized?          | Lower ↓     |
| <b>RMSE</b>          | What is the error magnitude on the target scale?  | Lower ↓     |
| <b>R<sup>2</sup></b> | How well does the model explain target variation? | Higher ↑    |

#### Computing the metrics

PYTHON

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f} RMSE: {rmse:.2f} R2: {r2:.3f}")
```

Always evaluate on `y_test` — data the model has never seen. Scoring on training data is like grading students on questions they already memorised.

**LEADING TO THE NEXT CONCEPT**

*"Numbers are useful, but a picture tells us more. Let's visualize predictions against reality."*

## SEE THE ERRORS

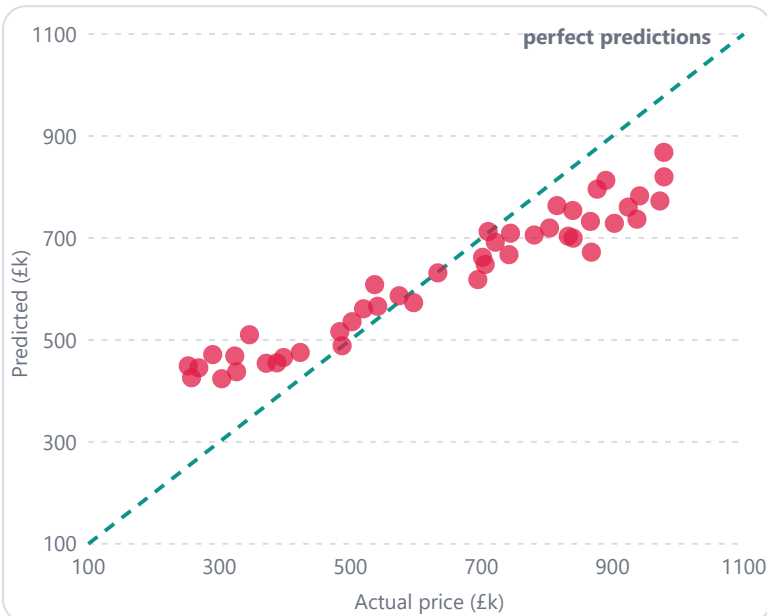
14

## Actual vs Predicted

Plot every test house: actual price on one axis, predicted price on the other. A perfect model puts every dot on the diagonal line.

### Compare a good model and a weak model

SIMULATED

 $R^2$   
**0.76**MAE  
**£96k**

Pattern bends away from the diagonal: cheaper flats are over-predicted and expensive houses under-predicted — a sign the model is missing something systematic.



**Good predictions tend to stay close to the diagonal relationship between actual and predicted values.**

Wide scatter → large errors. Dots consistently above or below the line → the model is systematically over- or under-predicting.

### Actual vs Predicted plot

PYTHON

```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")

plt.show()
```

▼ Tip: add the perfect-prediction diagonal

## Add a reference line

PYTHON

```
lims = [y_test.min(), y_test.max()]  
plt.plot(lims, lims, "r--") # dots on this line = perfect predictions
```

**LEADING TO THE NEXT CONCEPT**

*"What if the model performs very well on training data but poorly on new data?"*

15

## THE REAL GOAL

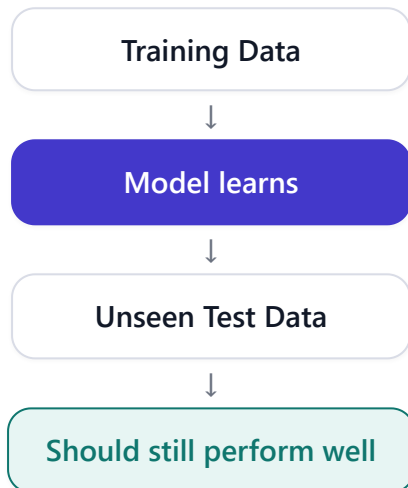
## Generalization



## WHY ARE WE LEARNING THIS NOW?

What if the model performs very well on training data but poorly on new data?

The client doesn't care how well the model fits *last year's* houses. They care about *next month's* houses.



## DEFINITION

**Generalization = ability of a model to perform well on unseen data.**

Like a student who understood the concepts vs one who memorised last year's paper.

Training vs Test performance ( $R^2$ )

|                |                        |      |
|----------------|------------------------|------|
| Training $R^2$ | <div><div></div></div> | 0.55 |
| Test $R^2$     | <div><div></div></div> | 0.52 |



**Underfitting.** Both scores are poor. The model is too simple to capture the pattern — it hasn't learned enough. Gap = 0.03



**Training  $R^2 = 0.99$ , Test  $R^2 = 0.65$ .** The model looks brilliant in the lab and disappoints in production. A big gap between training and test performance is the classic warning sign that the model has *memorised* instead of *learned*.



## LEADING TO THE NEXT CONCEPT

*"So why does a model end up too simple or too complex? Let's see both failure modes in action."*

16

## TWO WAYS TO FAIL

## Underfitting and Overfitting



## Underfitting

Model too simple



Misses important patterns



Poor training performance



Poor test performance



## Overfitting

Model too complex



Learns training data too closely



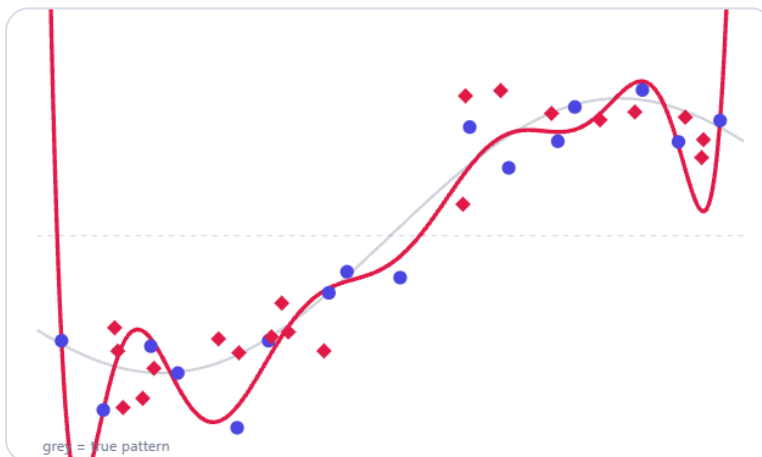
Excellent training performance



Poor test performance

## Interactive: drag model complexity

## POLYNOMIAL CURVE ON SIMULATED DATA



## Overfitting

TRAIN ERROR

**0.15**

TEST ERROR

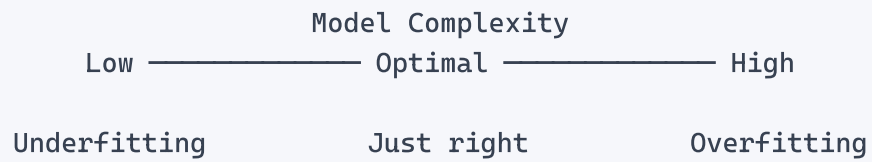
**0.41**

• training points • unseen test points

The curve wiggles through training dots to reduce training error, but swings wildly between them. Test error jumps → high variance.



Train error keeps falling as complexity grows. Test error falls, then rises again. The sweet spot is the bottom of the test curve.

**LEADING TO THE NEXT CONCEPT**

*"Underfitting and overfitting have names in ML theory: bias and variance. Let's connect them."*

## THE VOCABULARY

17

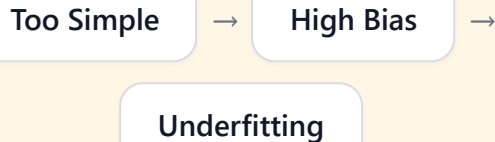
## Bias and Variance



## WHY ARE WE LEARNING THIS NOW?

You've *seen* underfitting and overfitting. Bias and variance are the words data scientists use to describe *why* they happen — you'll hear them in every model review.

## Bias = wrong assumptions



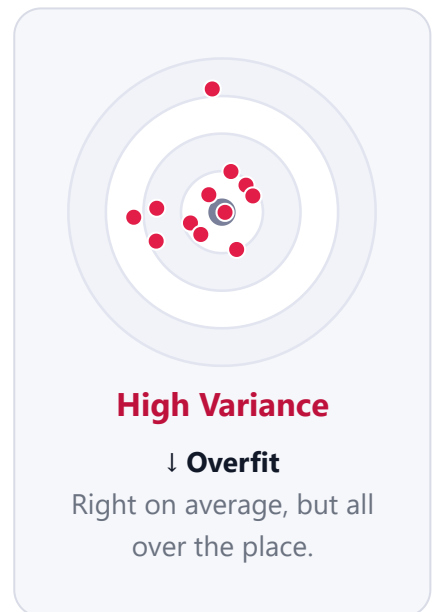
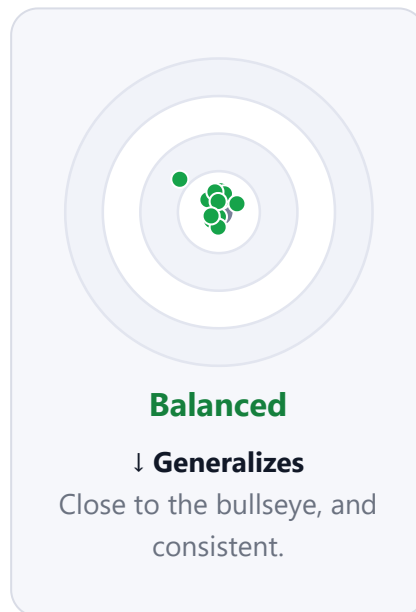
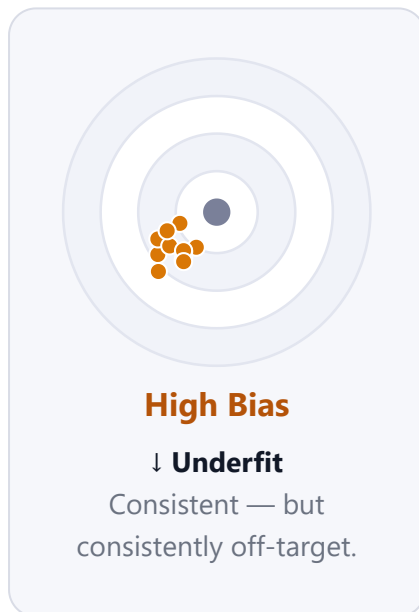
The model is "stubborn" — e.g. insisting on a straight line when the truth is curved. It's consistently wrong in the same direction.

## Variance = too sensitive



The model is "jumpy" — change a few training rows and it changes a lot. It chases noise.

## The dartboard view — each dot is a model trained on a slightly different sample



## THE BIAS-VARIANCE TRADE-OFF

Making a model more flexible **reduces bias** but **increases variance**. Making it simpler does the opposite. We can't drive both to zero — the goal is the **balance** that gives the lowest error on unseen data.

**⚡ CHECK 3**

**Training  $R^2 = 0.99$  and Test  $R^2 = 0.55$ . What is the likely concern?**

A. Underfitting

**B. Overfitting ✓ correct**

C. No correlation

D. Classification problem

**Answer: B. Overfitting** — Near-perfect training score but much lower test score = the model memorised the training data. That's overfitting (high variance).

**LEADING TO THE NEXT CONCEPT**

*"If excessive model complexity can cause overfitting, can we control the model's complexity?"*

## THE FIX

18

## Why Regularization?

**WHY ARE WE LEARNING THIS NOW?**

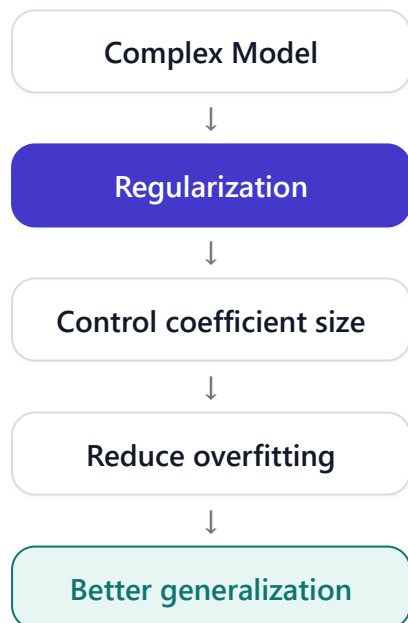
Our model may perform extremely well on training data but poorly on unseen data. If excessive model complexity causes overfitting, can we control the model's complexity?

In linear models, complexity shows up as **large coefficients**. A model that gives a huge weight to some feature is betting heavily on it — often because it's chasing noise in the training data.

**DEFINITION**

Regularization adds a penalty that discourages unnecessarily large model coefficients.

The model now has two goals: fit the data well **and** keep its coefficients modest. It only uses a big coefficient if the data really justifies it.

**An everyday analogy**

Think of a budget cap on a project team. Without a cap, every team member asks for more resources "just in case". With a cap, only the truly important work gets funded.

Regularization is a budget cap on coefficients. The strength of the cap is controlled by a setting called **alpha**.

**LEADING TO THE NEXT CONCEPT**

*"Regularization is the idea. Ridge is our first tool that puts it into practice."*

## L2 REGULARIZATION

19

# Ridge Regression

Linear Regression

+

L2 Regularization

=

Ridge

Linear Regression



Potentially large coefficients



Ridge



Shrink coefficients



Control complexity

**KEY POINT**

Ridge shrinks coefficients toward zero but usually does not make them exactly zero.

Every feature stays in the model — each just gets a quieter voice.

**alpha — the strength knob**

**Higher alpha** → **stronger regularization** → smaller coefficients.

alpha = 0 → plain Linear Regression. Very large alpha → coefficients squashed too much → underfitting.

**Ridge Regression**

PYTHON

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=1.0)

ridge.fit(X_train, y_train)

ridge_pred = ridge.predict(X_test)
```

Same `fit` / `predict` pattern. The only new thing is `alpha`.

**LEADING TO THE NEXT CONCEPT**

*"What if we don't just want smaller coefficients? What if some features are not useful at all?"*

## L1 REGULARIZATION

20

## Lasso Regression



## THE QUESTION

What if we don't just want smaller coefficients? What if some features are not useful at all?

Linear Regression

+

L1 Regularization

=

Lasso

Lasso



Shrink coefficients



Some coefficients can become zero



Feature-selection effect



## KEY INTUITION

Lasso can push some coefficients all the way to zero, which gives us a feature-selection effect.

A coefficient of 0 means the model ignores that feature completely.

## Lasso Regression

PYTHON

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.1)

lasso.fit(X_train, y_train)

lasso_pred = lasso.predict(X_test)
```

## Which features did Lasso keep?

PYTHON

```
pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": lasso.coef_
})
```

Coefficients close to or exactly zero show features whose contribution has been suppressed by the regularization. In the lab you'll see Lasso (with `alpha=2`) set a deliberately useless column (`random_noise`) to exactly 0. The right alpha depends on your data — that's why we experiment with it.

**⚡ CHECK 4**

**Which regularization can make coefficients exactly zero?**

A. Ridge

**B. Lasso ✓ correct**

**Answer: B. Lasso** — Lasso (L1) can push coefficients to exactly zero — that's its feature-selection effect. Ridge (L2) shrinks them but generally keeps them non-zero.



**Sketch pad — explain “Lasso Regression” by hand**

saved notes

Lasso

$$E = \|Y - \hat{Y}\|^2 + \lambda \|m\|$$

$$Y = m_1 x_1 + m_2 x_2 + m_3 x_3$$

~~zero~~  
~~10~~  
~~3~~

$$m_1 = 0.5$$

$$m_2 = 0.6$$

$$m_3 = 0$$

$$A = 4, D = 5, \text{age} = 10$$

Y

$$Y = 0.5 \times 4 + 0.6 \times 5 + 0 \times 10$$

Lasso

① overfitting  
 ② Feature selection



|

Lasso

$$E = \|y - \hat{y}\|^2 + \lambda(m)$$

$$y = m_1 x_1 + m_2 x_2 + m_3 x_3$$

~~zero~~

$$m_1 = 0.5$$

$$m_2 = 0.6$$

$$m_3 = 0$$

$$A = 4, D = 5, \text{age} = 10$$

Y

$$y = 0.5x_1 + 0.6x_2 + 0x_3$$

~~10~~

Lasso

① overfitting  
 → Feature selection



|

**LEADING TO THE NEXT CONCEPT**

*"We now have two regularization tools. Let's put them side by side and see exactly how they differ."*

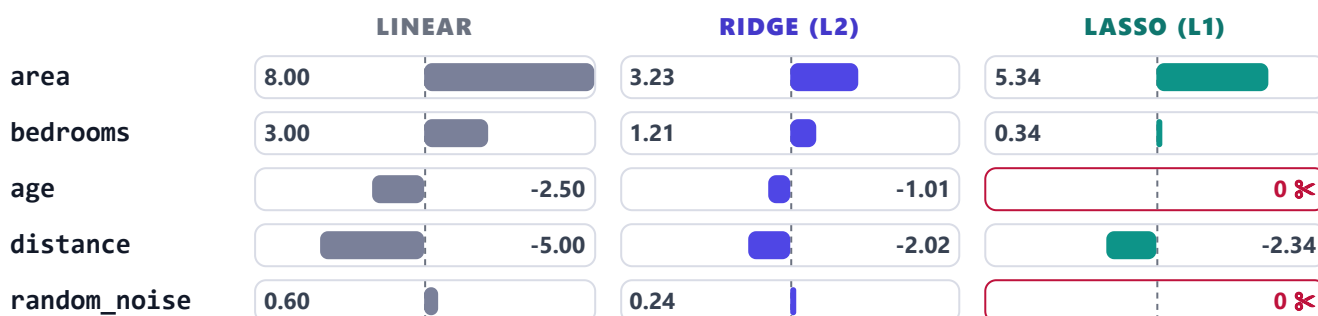
## SIDE BY SIDE

21

## Ridge vs Lasso

Turn up alpha and watch the coefficients

CONCEPTUAL ILLUSTRATION



**Ridge:** all 5 features still in the model, just smaller. **Lasso:** 2 features removed (coefficient = 0) — notice the weakest ones go first.

**Ridge**

"Keep the features, but reduce their influence."

**Lasso**

"Reduce their influence and potentially remove some features."

|                                            | RIDGE                                                       | LASSO                                                              |
|--------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Regularization</b>                      | L2                                                          | L1                                                                 |
| <b>Main effect</b>                         | Shrinks coefficients                                        | Shrinks coefficients                                               |
| <b>Can make coefficients exactly zero?</b> | Generally no                                                | Yes                                                                |
| <b>Feature-selection effect</b>            | No                                                          | Yes                                                                |
| <b>Basic intuition</b>                     | Keep features but reduce influence                          | Potentially remove less useful features                            |
| <b>Reach for it when...</b>                | Many features each contribute a little; correlated features | You suspect several features are useless; you want a simpler model |



## LEADING TO THE NEXT CONCEPT

"Enough theory. Time to run the whole story end-to-end in Python."

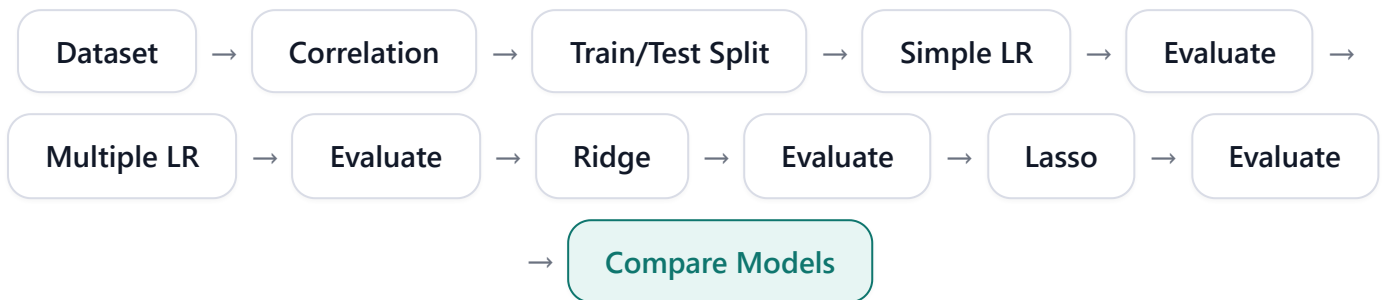
22

## COMPLETE WORKFLOW



## Hands-on Lab

Run the whole ML Module 1 story on one dataset, in a Jupyter notebook or Google Colab. Each block builds on the previous one — run them in order.



## Step 0 · Imports

## Step 0 — Setup

PYTHON

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

```

## Step 1 · Dataset

We generate a realistic **London housing dataset** so everyone gets identical results:

- **area** — floor area in sq ft (as quoted by UK estate agents)
- **bedrooms** — number of bedrooms
- **age** — age of the building in years (London has plenty of 100-year-old Victorian and Edwardian homes)
- **distance** — miles from central London (Charing Cross)
- **price** — sale price in **£ thousands** (533 = £533,000)

We also add a **random\_noise** column that has **nothing** to do with price — watch what Lasso does with it later.

## Step 1 — Create the London housing dataset

PYTHON

```

rng = np.random.default_rng(42)
n = 500

area = rng.normal(900, 300, n).clip(300, 2500).round() # sq ft
bedrooms = np.clip(np.round(area / 350 + rng.normal(0, 0.7, n)), 1, 6)

```

```

age = rng.uniform(0, 120, n).round()           # years
distance = rng.uniform(1, 20, n).round(1)      # miles from central London
random_noise = rng.normal(0, 1, n).round(2)    # a useless feature

price = (300 + 0.45 * area + 25 * bedrooms - 0.8 * age
        - 18 * distance + rng.normal(0, 45, n)) # £ thousands

df = pd.DataFrame({
    "area": area, "bedrooms": bedrooms, "age": age,
    "distance": distance, "random_noise": random_noise,
    "price": price.clip(150).round()
})

print(df.shape)
df.head()

```

### ▼ Expected output

```

(500, 6)
   area  bedrooms   age  distance  random_noise  price
0   991.0         4.0  78.0      10.1          0.39  638.0
1   588.0         2.0  41.0      15.9          0.55  264.0
2  1125.0         3.0  49.0      15.2          0.23  603.0
3  1182.0         2.0  53.0      16.5          0.97  508.0
4   315.0         1.0  15.0       9.5          0.09  333.0

```

Row 0: a 991 sq ft, 4-bed home, 78 years old, 10 miles out → £638k. The average price in this dataset is about £533k.

With your own client data, replace this block with `df = pd.read_csv("your_file.csv")`. A good real UK source to practise on is HM Land Registry's Price Paid Data.

## Step 2 • Correlation analysis

### Step 2 — Understand relationships first

PYTHON

```

corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.show()

corr["price"].sort_values(ascending=False)

```

### ▼ Expected output & what to notice

```

price      1.00
area       0.82

```

```
bedrooms      0.66
random_noise  -0.09
age           -0.10
distance      -0.56
```

Bigger homes cost more (area 0.82). The further from central London, the cheaper (distance  $-0.56$ ). Area and bedrooms are also correlated with each other ( $\sim 0.73$ ) — they overlap. `random_noise` is near 0, as expected.

## Step 3 · Train/Test split

### Step 3 — Hold back unseen data

PYTHON

```
features = ["area", "bedrooms", "age", "distance", "random_noise"]

X = df[features]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape)  # (400, 5) (100, 5)
```

## Step 4 · A reusable evaluation helper

### Step 4 — Fit, predict and score in one call

PYTHON

```
results = []

def evaluate(name, model, X_tr, X_te):
    model.fit(X_tr, y_train)
    y_pred = model.predict(X_te)
    scores = {
        "Model": name,
        "Train R2": round(model.score(X_tr, y_train), 3),
        "MAE": round(mean_absolute_error(y_test, y_pred), 2),
        "RMSE": round(float(np.sqrt(mean_squared_error(y_test, y_pred))), 2),
        "R2": round(r2_score(y_test, y_pred), 3),
    }
    results.append(scores)
    print(scores)
    return y_pred
```

We record **Train  $R^2$**  too, so we can spot a train-vs-test gap (overfitting). MAE and RMSE are in £ thousands.

## Step 5 · Simple Linear Regression → Evaluate

### Step 5 — One feature: area

PYTHON

```
simple = LinearRegression()  
simple_pred = evaluate("Simple LR (area)", simple, X_train[["area"]], X_test[["area"]])
```

#### ▼ Expected output

```
{'Model': 'Simple LR (area)', 'Train R2': 0.668, 'MAE': 87.54, 'RMSE': 105.72, 'R2': 0.688}
```

Typical miss  $\approx$  £88k. Area alone explains ~69% of the price variation — useful, but an estate agent wouldn't accept being £88k out.

## Step 6 · Multiple Linear Regression → Evaluate

### Step 6 — All features

PYTHON

```
lr = LinearRegression()  
lr_pred = evaluate("Multiple LR", lr, X_train, X_test)  
  
plt.scatter(y_test, lr_pred, alpha=0.6)  
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--")  
plt.xlabel("Actual (£k)"); plt.ylabel("Predicted (£k)"); plt.title("Actual vs Predicted")  
plt.show()
```

#### ▼ Expected output

```
{'Model': 'Multiple LR', 'Train R2': 0.949, 'MAE': 41.79, 'RMSE': 48.66, 'R2': 0.934}
```

A big jump: MAE halves from £88k to £42k once the model also sees bedrooms, age and distance from central London. Train (0.949) and test (0.934)  $R^2$  are close → it generalizes well.

## Step 7 · Ridge → Evaluate

### Step 7 — Ridge (L2)

PYTHON

```
ridge = Ridge(alpha=1.0)  
ridge_pred = evaluate("Ridge (alpha=1)", ridge, X_train, X_test)
```

#### ▼ Expected output

```
{'Model': 'Ridge (alpha=1)', 'Train R2': 0.949, 'MAE': 41.8, 'RMSE': 48.67, 'R2': 0.934}
```

## Step 8 • Lasso → Evaluate

### Step 8 — Lasso (L1) and its coefficients

PYTHON

```
lasso = Lasso(alpha=2.0)
lasso_pred = evaluate("Lasso (alpha=2)", lasso, X_train, X_test)

coefs = pd.DataFrame({
    "Feature": X.columns,
    "Linear": lr.coef_,
    "Ridge": ridge.coef_,
    "Lasso": lasso.coef_
}).round(3)

coefs
```

#### ▼ Expected output

```
{'Model': 'Lasso (alpha=2)', 'Train R2': 0.949, 'MAE': 42.15, 'RMSE': 49.06, 'R2': 0.933}
```

|   | Feature      | Linear  | Ridge   | Lasso   |
|---|--------------|---------|---------|---------|
| 0 | area         | 0.467   | 0.467   | 0.478   |
| 1 | bedrooms     | 21.856  | 21.754  | 18.069  |
| 2 | age          | -0.754  | -0.754  | -0.748  |
| 3 | distance     | -17.658 | -17.657 | -17.623 |
| 4 | random_noise | -1.435  | -1.429  | -0.000  |

Read the coefficients like a London estate agent: each extra sq ft adds ~£470, each bedroom ~£22k, each year of age takes off ~£750, and every mile further from central London takes off ~£18k.

Lasso set `random_noise` to exactly zero (shown as `-0.000`) — it removed the useless feature. Ridge only nudged it smaller.

## Step 9 • Compare models

### Step 9 — Results table

PYTHON

```
comparison = pd.DataFrame(results).set_index("Model")
comparison
```

#### ▼ Expected output

|                  | Train R2 | MAE   | RMSE   | R2    |
|------------------|----------|-------|--------|-------|
| Model            |          |       |        |       |
| Simple LR (area) | 0.668    | 87.54 | 105.72 | 0.688 |
| Multiple LR      | 0.949    | 41.79 | 48.66  | 0.934 |
| Ridge (alpha=1)  | 0.949    | 41.80 | 48.67  | 0.934 |
| Lasso (alpha=2)  | 0.949    | 42.15 | 49.06  | 0.933 |

## Step 10 · Experiment: what happens when alpha increases?

### Step 10 — Turn up Lasso's alpha

PYTHON

```
for alpha in [0.1, 2, 25, 500, 2000]:
    m = Lasso(alpha=alpha).fit(X_train, y_train)
    kept = [f for f, c in zip(features, m.coef_) if c != 0]
    print(f"alpha={alpha:<5} R2={r2_score(y_test, m.predict(X_test)):.3f} kept: {kept}")
```

#### ▼ Expected output

```
alpha=0.1  R2=0.934  kept: ['area', 'bedrooms', 'age', 'distance', 'random_noise']
alpha=2    R2=0.933  kept: ['area', 'bedrooms', 'age', 'distance']
alpha=25   R2=0.922  kept: ['area', 'age', 'distance']
alpha=500  R2=0.727  kept: ['area', 'age', 'distance']
alpha=2000 R2=0.683  kept: ['area']
```

As alpha rises, Lasso drops more features (first the useless one, then bedrooms, eventually everything except area) and  $R^2$  falls — too much regularization leads to **underfitting**.



#### TRAINER'S NOTE

In real projects we usually **scale features** (put them on a similar range) before Ridge/Lasso, because the penalty treats every coefficient equally. Here, **area** runs into the hundreds and thousands of sq ft while **bedrooms** is 1–6, which is partly why Lasso drops bedrooms early. We'll keep it simple in this module.



#### LEADING TO THE NEXT CONCEPT

*"You have the numbers. Now let's read them like a consultant and decide which model wins."*

## READ THE RESULTS

23

## Model Comparison

Type in your lab results (or load the reference values from Step 9), then highlight the winners.

| MODEL             | MAE ↓ | RMSE ↓ | R <sup>2</sup> ↑ |
|-------------------|-------|--------|------------------|
| Linear Regression | 41.79 | 48.66  | 0.934            |
| Ridge             | 41.8  | 48.67  | 0.934            |
| Lasso             | 42.15 | 49.06  | 0.933            |



Best per metric — **MAE**: Linear Regression (differences are tiny) · **RMSE**: Linear Regression (differences are tiny) · **R<sup>2</sup>**: Linear Regression & Ridge (differences are tiny)  
When scores are this close, pick the simpler, more explainable model.

### Investigate like a consultant

#### QUESTION 1

Which model has the lowest MAE (reference run)?

A. Linear Regression (Ridge is a hair behind) ✓ correct

B. Lasso

C. Simple LR (area only)

**Answer: A. Linear Regression (Ridge is a hair behind)** — Linear Regression has MAE 41.79, Ridge 41.80, Lasso 42.15 (all in £k). Linear wins by £10 — practically a tie with Ridge. Lasso is about £360 worse on average: still tiny next to a typical £533k price.

 **QUESTION 2**

**Which model has the lowest RMSE?**

A. Lasso, clearly

**B. Linear & Ridge ( $\approx$  £48.7k), by a hair ✓ correct**

C. Simple LR

**Answer: B. Linear & Ridge ( $\approx$  £48.7k), by a hair** — Linear 48.66, Ridge 48.67, Lasso 49.06 (£k). A difference of £10 between Linear and Ridge is noise, not a real win.

 **QUESTION 3**

**Which model has the highest  $R^2$ ?**

A. Linear Regression by a wide margin

B. Lasso by a wide margin

**C. They are practically tied ( $\sim 0.934$ ) ✓ correct**

**Answer: C. They are practically tied ( $\sim 0.934$ )** — All three score  $R^2 \approx 0.933$ – $0.934$ . When metrics are this close, prefer the simpler or more explainable model.

 **QUESTION 4**

**Did regularization improve test performance in this dataset?**

A. Yes, a big improvement

**B. Not really — the base model wasn't overfitting ✓ correct**

C. It made the model much worse

**Answer: B. Not really — the base model wasn't overfitting** — Here the plain model wasn't overfitting (train  $R^2$  0.949 vs test 0.934 — a small gap) and we have 400 rows for 5 features. Regularization helps most when there's a large train–test gap, many features, or few rows. That's an important, honest lesson: it's a tool for overfitting, not a magic accuracy boost.

 **QUESTION 5**

**Which feature did Lasso (alpha=2) reduce to zero?**

A. area

B. bedrooms

C. distance

**D. random\_noise ✓ correct**

**Answer: D. random\_noise** — Lasso (alpha=2) set random\_noise to exactly 0 — the feature we deliberately created with no relationship to price. That's the feature-selection effect working as intended.

 **QUESTION 6**

**What happens when alpha increases a lot?**

A. More features are kept and accuracy always improves

B. Nothing changes

**C. More coefficients become zero; too much → underfitting ✓ correct**

**Answer: C. More coefficients become zero; too much → underfitting** — Higher alpha = stronger penalty: more coefficients shrink to zero (alpha=25 drops bedrooms, alpha=2000 keeps only area), and  $R^2$  falls (0.727 at alpha=500, 0.683 at 2000) — underfitting.

**LEADING TO THE NEXT CONCEPT**

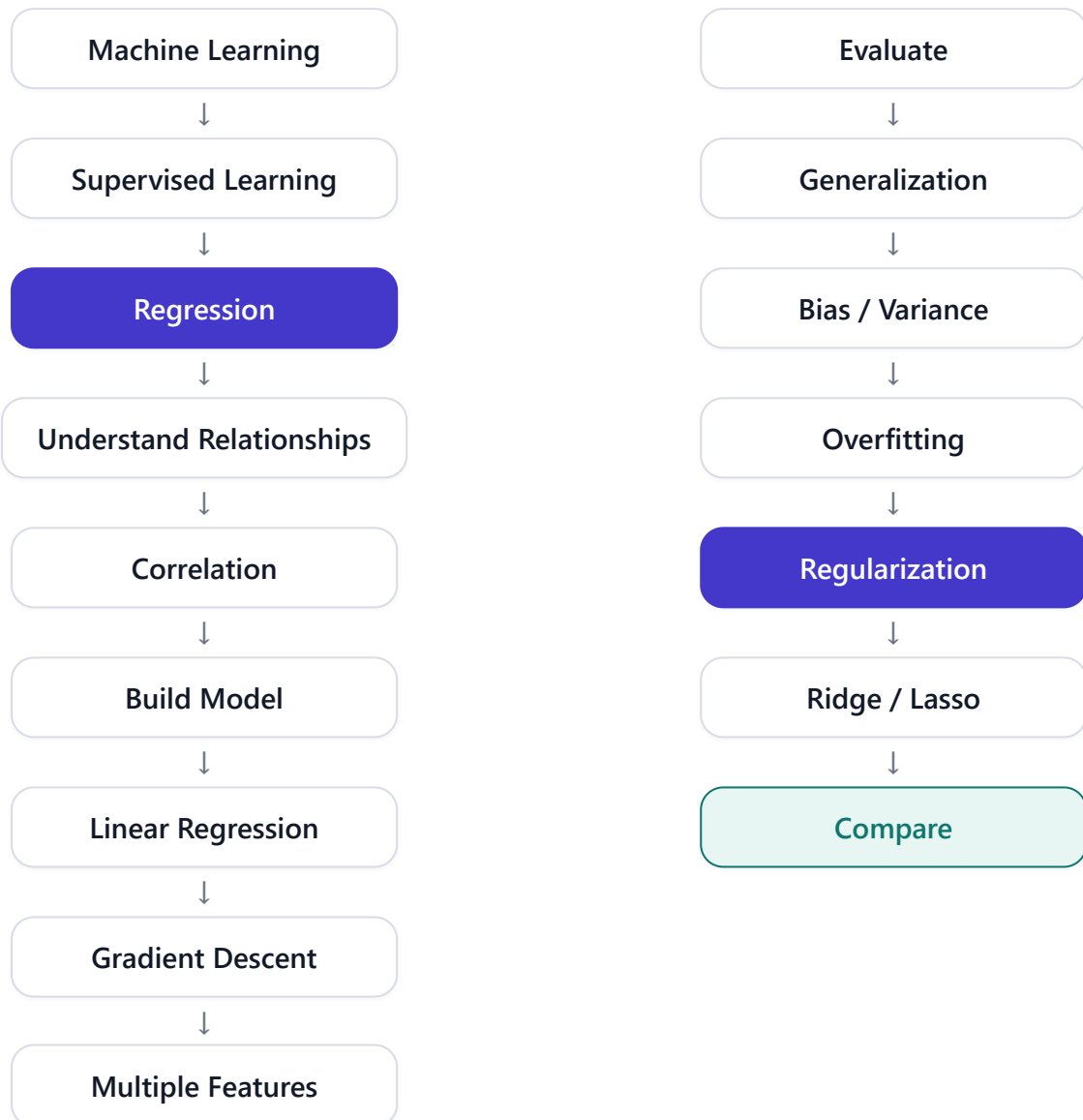
*"Let's step back and see the whole story we told in this module."*

24

THE STORY SO FAR

## ML Module 1 Recap

Everything in this module was one story — each step answered the question raised by the step before.



### What should I be able to explain after ML Module 1?

Tick each one you could explain to a colleague in under a minute. **0 / 17**

☐ What Machine Learning is☐ Supervised vs Unsupervised vs Reinforcement Learning☐ Regression vs Classification☐ The basic ML workflow☐ What correlation means☐ Positive vs negative correlation

☐ Correlation vs causation☐ What a regression problem is☐ Simple vs Multiple Linear Regression☐ How gradient descent trains a linear regression (and what the learning rate does)☐ MAE, RMSE and  $R^2$ ☐ Generalization☐ Bias and Variance☐ Underfitting and Overfitting☐ Why regularization is needed☐ Ridge vs Lasso☐ Why Lasso can perform feature selection



# ML Module 1 Complete

You started with the fundamentals of Machine Learning, understood where Regression fits, explored relationships using Correlation, built Linear Regression models, evaluated them, understood generalization and overfitting, and finally learned how Ridge and Lasso help control model complexity.